

```

1  """Week 2 DSP + ML teaching demos: sampling, aliasing, reconstruction, ZOH, quantization.
2
3  Designed for a combined senior / first-year graduate ECE course using
4  Oppenheim & Schaffer, Discrete-Time Signal Processing.
5
6  Run as a script to generate figures and WAV files in ./generated_week2.
7  The companion notebook presents the same ideas interactively.
8  """
9  from __future__ import annotations
10
11  from pathlib import Path
12  import math
13  import numpy as np
14  import matplotlib.pyplot as plt
15  from scipy import signal
16  from scipy.io import wavfile
17
18  try:
19      import torch
20  except ImportError: # keep most demos usable without torch
21      torch = None
22
23
24  def normalized_sinc(x: np.ndarray) -> np.ndarray:
25      """sinc(x) = sin(pi x)/(pi x), with sinc(0)=1."""
26      return np.sinc(x)
27
28
29  def sample_sinusoid(f0_hz: float = 900.0, fs_hz: float = 4000.0, duration_s: float =
30  0.006):
31      """Dense 'continuous-time' grid plus ideal sample values."""
32      t = np.linspace(0.0, duration_s, 3000, endpoint=False)
33      x_c = np.cos(2 * np.pi * f0_hz * t)
34      n = np.arange(int(np.floor(duration_s * fs_hz)))
35      t_n = n / fs_hz
36      x_n = np.cos(2 * np.pi * f0_hz * t_n)
37      return t, x_c, n, t_n, x_n
38
39  def alias_to_baseband(f_hz: float, fs_hz: float) -> float:
40      """Fold a real sinusoid frequency into [0, fs/2]."""
41      f_mod = np.mod(f_hz, fs_hz)
42      return float(min(f_mod, fs_hz - f_mod))
43
44
45  def sampled_spectrum_copies(freq: np.ndarray, baseband_center_hz: float, bandwidth_hz:
46  float,
47      fs_hz: float, kmax: int = 2) -> np.ndarray:
48      """Simple triangular CT spectrum and its sampled replicas (teaching
49      visualization)."""
50      def tri(z):
51          return np.maximum(1.0 - np.abs(z - baseband_center_hz) / bandwidth_hz, 0.0) + \
52              np.maximum(1.0 - np.abs(z + baseband_center_hz) / bandwidth_hz, 0.0)
53      out = np.zeros_like(freq, dtype=float)
54      for k in range(-kmax, kmax + 1):
55          out += tri(freq - k * fs_hz)
56      return out
57
58  def synth_voice_like(fs_hz: int = 48000, duration_s: float = 2.5) -> np.ndarray:
59      """Deterministic voice-like waveform with harmonics + an HF component for alias
60      demos."""
61      t = np.arange(int(fs_hz * duration_s)) / fs_hz
62      # Slowly varying F0 around 180 Hz.
63      f0 = 180.0 + 15.0 * np.sin(2 * np.pi * 2.2 * t)
64      phase = 2 * np.pi * np.cumsum(f0) / fs_hz
65      x = np.zeros_like(t)

```

```

64     # Harmonic stack with a smooth spectral tilt.
65     for k in range(1, 35):
66         x += (1.0 / k**1.15) * np.sin(k * phase + 0.15 * k)
67     # Formant-like shaping via resonant bandpass filters.
68     sos1 = signal.iirpeak(700 / (fs_hz / 2), Q=5)
69     sos2 = signal.iirpeak(1500 / (fs_hz / 2), Q=7)
70     sos3 = signal.iirpeak(2600 / (fs_hz / 2), Q=9)
71     shaped = 0.8 * signal.lfilter(*sos1, x) + 0.55 * signal.lfilter(*sos2, x) + 0.35 *
        signal.lfilter(*sos3, x)
72     # Add a controlled high-frequency component that will alias after 4x decimation.
73     hf = 0.16 * np.sin(2 * np.pi * 10000.0 * t)
74     # Syllabic amplitude envelope.
75     env = 0.25 + 0.75 * (0.5 + 0.5 * np.sin(2 * np.pi * 2.8 * t))**1.8
76     y = env * shaped + hf
77     y /= np.max(np.abs(y)) + 1e-12
78     return 0.9 * y
79
80
81 def decimate_naive(x: np.ndarray, factor: int) -> np.ndarray:
82     return x[::factor]
83
84
85 def decimate_antialias(x: np.ndarray, factor: int) -> np.ndarray:
86     # Polyphase FIR performs anti-aliasing and decimation.
87     return signal.resample_poly(x, up=1, down=factor)
88
89
90 def sinc_reconstruct(xn: np.ndarray, fs_hz: float, t_eval: np.ndarray, n0: int = 0) -> np
    .ndarray:
91     """Finite sinc interpolation from a finite sequence of samples."""
92     n = np.arange(n0, n0 + len(xn))
93     tau = t_eval[:, None] * fs_hz - n[None, :]
94     return normalized_sinc(tau) @ xn
95
96
97 def zoh_waveform(xn: np.ndarray, fs_hz: float, oversample: int = 30):
98     """Simple zero-order hold visualization."""
99     y = np.repeat(xn, oversample)
100    t = np.arange(len(y)) / (fs_hz * oversample)
101    return t, y
102
103
104 def uniform_quantize(x: np.ndarray, bits: int, full_scale: float = 1.0):
105     """Mid-tread uniform quantizer over approximately [-full_scale, full_scale]."""
106     if bits < 1:
107         raise ValueError("bits must be >= 1")
108     levels = 2**bits
109     delta = 2 * full_scale / levels
110     x_clip = np.clip(x, -full_scale, full_scale - delta)
111     q = delta * np.round(x_clip / delta)
112     q = np.clip(q, -full_scale, full_scale - delta)
113     return q, delta
114
115
116 def measured_sqr_db(x: np.ndarray, xq: np.ndarray) -> float:
117     noise = xq - x
118     return 10 * np.log10(np.mean(x**2) / np.mean(noise**2))
119
120
121 def theoretical_sqr_db(bits: int) -> float:
122     return 6.02 * bits + 1.76
123
124
125 def stft_power_db(x: np.ndarray, fs_hz: float, nperseg: int = 1024):
126     f, t, Z = signal.stft(x, fs=fs_hz, nperseg=nperseg, noverlap=3*nperseg//4,
        window="hann", boundary=None)
127     p = 20 * np.log10(np.maximum(np.abs(Z), 1e-8))

```

```

129     return f, t, p
130
131
132 def torch_stft_feature(x: np.ndarray, n_fft: int = 1024, hop: int = 256):
133     """Magnitude STFT using PyTorch, demonstrating float conversion for ML."""
134     if torch is None:
135         raise RuntimeError("PyTorch is not installed")
136     xt = torch.as_tensor(x, dtype=torch.float32)
137     window = torch.hann_window(n_fft)
138     X = torch.stft(xt, n_fft=n_fft, hop_length=hop, window=window, return_complex=True)
139     return X.abs()
140
141
142 def int16_to_float32(x_int16: np.ndarray) -> np.ndarray:
143     """Typical audio-to-ML conversion: int16 PCM to roughly [-1, 1)."""
144     if x_int16.dtype != np.int16:
145         raise TypeError("expected int16 PCM")
146     return x_int16.astype(np.float32) / 32768.0
147
148
149 def write_wav(path: Path, fs_hz: int, x: np.ndarray):
150     x = np.asarray(x)
151     peak = np.max(np.abs(x)) + 1e-12
152     if peak > 1.0:
153         x = x / peak
154     pcm = np.int16(np.clip(x, -1, 1 - 1/32768) * 32767)
155     wavfile.write(path, fs_hz, pcm)
156
157
158 def make_demo_outputs(outdir: Path) -> None:
159     outdir.mkdir(parents=True, exist_ok=True)
160     fs = 48000
161     x = synth_voice_like(fs)
162     factor = 4
163     fs_ds = fs // factor
164     x_naive = decimate_naive(x, factor)
165     x_aa = decimate_antialias(x, factor)
166     xq8, _ = uniform_quantize(x, 8)
167     xq4, _ = uniform_quantize(x, 4)
168
169     write_wav(outdir / "01_clean_48k.wav", fs, x)
170     write_wav(outdir / "02_decimated_naive_12k_alias.wav", fs_ds, x_naive)
171     write_wav(outdir / "03_decimated_antialias_12k.wav", fs_ds, x_aa)
172     write_wav(outdir / "04_quantized_8bit.wav", fs, xq8)
173     write_wav(outdir / "05_quantized_4bit.wav", fs, xq4)
174
175     print(f"Wrote demo WAV files to {outdir}")
176     print(f"10 kHz at fs={fs_ds} Hz aliases to {alias_to_baseband(10000, fs_ds):.0f} Hz")
177     for b in [4, 8, 12, 16]:
178         t = np.arange(fs) / fs
179         sine = 0.999 * np.sin(2*np.pi*997*t)
180         q, _ = uniform_quantize(sine, b)
181         print(f"B={b:2d}: theory={theoretical_sqr_db(b):6.2f} dB, measured={
182             measured_sqr_db(sine,q):6.2f} dB")
183
184 if __name__ == "__main__":
185     make_demo_outputs(Path(__file__).with_name("generated_week2"))
186

```